“AuraFit”: Exercise Scheduler

Resource-Constrained Fitness Program Design

Final Report
Advanced Algorithms
Prepared by: Aaron Tobias
Date: May 03, 2026

Repository: https://github.com/Throwbias/Fitness-Scheduler

Abstract

This report presents “AuraFit”, a fitness scheduling system that formulates weekly resistance-training design as a resource-constrained optimization problem. The system models a seven-day training routine as a chromosome whose genes are daily exercise sessions. Candidate schedules are evaluated by a multi-component fitness function that rewards movement-category coverage, training density, high-priority exercise selection, and fatigue balance while penalizing time-limit violations, excessive daily fatigue, repeated exercises, and insufficient recovery spacing. The implemented system uses a SQLite-backed exercise database, a JSON planning request, dataclass-based schedule objects, tournament selection, elitism, uniform day-level crossover, and targeted mutation operators. Experimental evaluation compares the genetic algorithm and refinement-oriented schedule generation against greedy and random baselines, showing that globally evaluated plans produce more balanced schedules than purely sequential construction. The project aims to demonstrate mastery of core advanced-algorithm concepts, including asymptotic analysis, greedy heuristics, randomized algorithms, local search, evolutionary computation, constraint handling, graph-like resource conflicts, data-structure design, and empirical benchmarking. Although the current prototype uses a modest exercise database and hand-tuned scoring weights, it establishes a defensible architecture for future extensions such as multi-week progression, personalized contraindication filters, equipment-aware planning, Pareto optimization, and larger-scale performance testing.

Table of Contents

1. Introduction and Motivation

AuraFit was developed to address a practical scheduling problem that appears simple at the surface but becomes algorithmically difficult once realistic training constraints are introduced. A person rarely needs only a list of good exercises. A usable training plan must place those exercises across time while respecting recovery, fatigue, equipment, session length, movement-pattern coverage, and user availability. The result is a scheduling problem rather than a simple recommendation problem. In this report, the weekly training plan is treated as a seven-day optimization domain in which each candidate solution must balance competing objectives.

Human training schedules resemble classical resource-constrained scheduling problems. Time is limited, fatigue is a renewable but bounded resource, movement categories represent coverage requirements, and muscle groups impose spacing constraints similar to resource conflicts. A schedule that maximizes exercise priority alone may be physiologically poor if it clusters several heavy lower-body movements on adjacent days. Conversely, a schedule that avoids fatigue entirely may be safe but under-dosed. This tension motivates a heuristic optimization approach rather than a one-pass deterministic rule.

The project uses a genetic algorithm because the search space is discrete, combinatorial, and multi-objective. For a library of n exercises and a seven-day schedule with up to k exercises per available day, the number of possible schedules grows rapidly. Exhaustive search is not practical even for moderate values of n and k. Greedy selection is efficient, but it makes local decisions before the full weekly pattern is visible. AuraFit instead evaluates complete weekly microcycles, allowing the algorithm to discover tradeoffs across days.

The system is also motivated by the course emphasis on moving from algorithmic theory to applied computational problem solving. Chapters 1-14 collectively cover asymptotic analysis,

divide-and-conquer reasoning, graph traversal, dynamic programming, randomized algorithms, approximation, complexity, and empirical evaluation. AuraFit integrates these ideas into a single domain-specific system: it uses formal representations, heuristic search, randomized exploration, careful objective-function design, testable data structures, and reproducible performance artifacts.

1.1 Project Contributions

The first contribution is a formal representation of workout planning as a constrained optimization problem. Instead of treating a workout as an isolated daily decision, the system encodes the full week as a chromosome. This makes recovery and fatigue balance first-class algorithmic properties.

The second contribution is a multi-component fitness function that translates qualitative training goals into measurable scores and penalties. The evaluator combines priority, density, coverage, fatigue balance, recovery spacing, and hard-constraint penalties into a single utility score.

The third contribution is a working implementation that connects algorithm design to system architecture. The project includes a data layer, request loader, genetic engine, evaluator, mutation and crossover operators, console output, tests, and generated experiment artifacts.

The fourth contribution is an empirical comparison to baselines. The report evaluates the genetic/refined scheduler against random and greedy schedule construction and explains where global search improves over local decision-making.

2. Related Work and Literature Review

Genetic algorithms are adaptive search procedures inspired by selection, recombination, and mutation. Holland introduced the theoretical foundations for adaptation in artificial systems, while Goldberg popularized genetic algorithms as practical search and optimization methods. Their central appeal is that they can search large, rugged spaces without requiring gradients or closed-form objective functions. AuraFit follows this tradition by representing schedules as chromosomes and repeatedly selecting, crossing, mutating, and scoring individuals.

Resource-constrained project scheduling provides a useful theoretical frame. In the classical RCPSP, activities must be placed in time under precedence and resource constraints. Fitness scheduling does not have identical precedence constraints, but it shares the core challenge: limited resources must be allocated across a time horizon while maintaining feasibility. Hartmann showed that genetic encodings can be competitive for RCPSP when they incorporate problem-specific knowledge. AuraFit applies the same principle by using a day-level encoding that preserves session structure during crossover.

Multi-objective optimization is also relevant because fitness programming involves multiple desirable properties that cannot be reduced naturally to one dimension. NSGA-II, introduced by Deb, Pratap, Agarwal, and Meyarivan, is a major reference point for elitist multi-objective evolutionary search. AuraFit currently uses a scalarized weighted fitness function rather than Pareto sorting, but the conceptual problem is multi-objective: users care about coverage, fatigue, recovery, priority, and time utilization at the same time.

Greedy algorithms provide an important baseline. A greedy scheduler can select the best feasible next exercise according to a local score, but it is vulnerable to early choices that create poor global balance. This is the same general limitation discussed in algorithm design texts such as Cormen et al. and Kleinberg and Tardos: greedy algorithms require strong structural properties to

guarantee optimality. Workout planning lacks those guarantees because an exercise that is locally valuable today can make tomorrow infeasible or redundant.

Exercise-science literature motivates the domain constraints. Resistance-training progression models emphasize progressive overload, specificity, variation, and recovery. Hypertrophy research also highlights the importance of training volume, frequency, and workload distribution. "AuraFit" does not claim to replace professional coaching, but it encodes several defensible programming principles: sessions should not exceed realistic fatigue caps, major movement patterns should be covered, repeated high-stress movement should be spaced, and weekly distribution should avoid extreme spikes.

2.1 Literature Mapping to System Design

Research Area	**Concept Used**	**AuraFit Implementation**
Genetic algorithms	Population search, crossover, mutation, elitism	GeneticSolver, GAOperators
RCPSP	Scheduling with limited resources	Time limit, fatigue cap, recovery spacing
Greedy algorithms	Baseline constructive heuristic	Greedy comparison and discussion
Local search	Neighborhood improvement after construction	Refinement-oriented analysis and future work
Resistance training models	Recovery, progression, coverage	Movement categories and fatigue penalties
Empirical algorithmics	Benchmarking and scaling	Runtime charts, baseline comparison, stability discussion

3. Problem Definition

The input to the problem is an exercise set E, a seven-day planning horizon D, and a user planning request R. Each exercise e in E has a category, muscle group, difficulty, duration, fatigue cost, priority, minimum recovery interval, and optional goal tags. The request contains available training days, a maximum session time, required movement categories, a daily fatigue cap, a maximum number of exercises per session, and a desired number of training days.

The output is a weekly schedule S in which each day d contains an ordered or unordered list of exercises. The current implementation treats intra-session exercise order as outside the optimization target; therefore the schedule is primarily a day-assignment problem. The output should be feasible with respect to user constraints and high-value with respect to the fitness objective.

The objective is not a single classical minimization such as makespan. Instead, the scheduler maximizes a utility function that combines soft rewards and hard penalties. The best schedule is the candidate with the highest total fitness after evaluating exercise priority, category coverage, fatigue distribution, density, recovery spacing, and violations.

3.1 Formalization

Let E = {e1, e2, ..., en} be the exercise library and D = {0,1,2,3,4,5,6} be the days of the week. Let D = {1, 2, ..., 7} denote the weekly planning horizon, and let E denote the set of available exercises. A candidate schedule is a function S: D → 2^E, where S(d) ⊆ E is the set of exercises assigned to day d. A day d is considered active if S(d) ≠ ∅ and inactive otherwise. The feasible subset is constrained by available days A, session time limit T, daily fatigue cap F, desired training frequency q, and maximum session size k.

For each day d, total time is time(d) = sum duration(e) for all e in S(d), and total fatigue is fatigue(d) = sum fatigue_cost(e) for all e in S(d). A hard violation occurs when time(d) > T,

fatigue(d) > F, duplicate exercises are assigned within the same day, or the number of active days deviates substantially from q. A recovery violation occurs when two exercises that stress the same muscle group, or two high-priority lifts, are closer than the required minimum recovery interval.

3.2 Constraints

Constraint	Definition	Treatment
Available days	Only user-approved day indices may contain exercises	Hard
Session time	Sum of durations should not exceed the time limit	Hard penalty
Fatigue cap	Sum of fatigue costs should not exceed daily cap	Hard penalty
Training frequency	Active days should match desired weekly frequency	Hard penalty
Movement coverage	Required categories should appear at least once	Soft reward / severe missing penalty
Recovery spacing	Conflicting muscle groups must be separated	Penalty
Exercise repetition	Repeated weekly exercise use is discouraged	Penalty
Session density	Useful time utilization is rewarded	Soft reward

4. Algorithms and Data Structures

The central data structure is the Individual object. It contains a chromosome represented as a seven-slot list, where each slot is a list of Exercise objects. This representation is compact, easy to mutate, and directly aligned with the planning horizon. It also avoids an unnecessary mapping layer because day index 0 corresponds to Monday and day index 6 corresponds to Sunday.

The Exercise and PlanningRequest classes are implemented as dataclasses. This is a strong design choice because the algorithm requires lightweight, immutable-in-practice records that can be copied and evaluated repeatedly. The Individual object also stores a fitness score and

a metrics dictionary for future analysis. The design cleanly separates problem data from optimization behavior.

4.1 Chromosomal Encoding

"AuraFit" uses direct schedule encoding. A chromosome is not a binary string; it is a structured weekly plan. Each gene corresponds to one day, and the value of the gene is a session list. This encoding is more expressive than a flat vector because crossover can preserve daily training blocks. Preserving blocks matters because a session often has internal logic: pushing and pulling movements may complement each other, while excessive lower-body duplication within one session may be undesirable.

Field	Gene 0	Gene 1	Gene 2	Gene 3	Gene 4	Gene 5	Gene 6
Day index	0	1	2	3	4	5	6
Meaning	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
Gene value	List[Exercise]	List[Exercise]	List[Exercise]	List[Exercise]	List[Exercise]	List[Exercise]	List[Exercise]

4.2 Genetic Algorithm

The genetic algorithm initializes a population of random schedules, evaluates them, preserves elite individuals, and fills the next generation with crossover and mutation. Parent selection uses tournament selection. For each tournament, a small random sample is drawn and the best individual in that sample becomes a parent. This balances selective pressure with diversity; the population is biased toward high-quality schedules without deterministically selecting only the current best.

The implemented process is summarized below:

Algorithm 1: Evolutionary scheduling procedure

1. Generate an initial population of random weekly schedules.

2. For each generation, evaluate every individual using the fitness function.

3. Sort individuals by fitness score.

4. Copy the top 5 percent into the next generation as elites.

5. Repeatedly select two parents using tournament selection.

6. Apply uniform day-level crossover to produce one child.

7. Apply mutation to the child with probability 0.1.

8. Replace the old population with the new population.

9. After the final generation, return the highest-scoring individual.

4.3 Crossover and Mutation

Crossover is uniform at the day level. For each available day, the child inherits the complete session from either parent. This is a problem-specific choice. A lower-level crossover that randomly mixes individual exercises might destroy useful session structure, whereas day-level crossover treats each session as a modular building block. This is consistent with the project goal of preserving physiologically coherent training blocks while exploring new weekly arrangements.

Mutation introduces diversity and prevents stagnation. The point mutation replaces one exercise with another from the exercise pool. Relocation mutation moves an exercise from one training day to another available day. Swap mutation exchanges exercises between two days. A targeted frequency correction can remove excess active days when a candidate schedule exceeds the desired weekly training frequency. Together, these operators allow the algorithm to explore selection, placement, and balance decisions.

4.4 Fitness Function

The evaluator converts a candidate weekly plan into a single scalar fitness score. The score can be understood as total_fitness = priority_score + balance_score + coverage_score +

density_bonus - spacing_penalty - violation_penalty. This scalarization makes optimization simpler than a true Pareto approach but still captures multiple objectives.

Component	Computation Intent	Effect
Priority score	Rewards high-priority exercises weighted by duration	Encourages valuable compound movements
Balance score	Rewards low deviation from average daily fatigue	Flattens weekly workload
Coverage score	Rewards required movement categories	Prevents narrow programs
Density bonus	Rewards sessions above 80 percent time utilization	Avoids under-dosed schedules
Spacing penalty	Penalizes insufficient recovery between conflicts	Protects recovery windows
Violation penalty	Penalizes time, fatigue, frequency, and repetition failures	Maintains feasibility

4.5 Complexity Analysis

Let P be the population size, G be the number of generations, D be the number of days in the planning horizon, K be the maximum exercises per session, and M be the number of scheduled exercises in a candidate. Initial population generation is O(PDK), assuming random sampling is bounded by K. Fitness evaluation includes scanning all sessions and performing pairwise recovery checks among scheduled exercises. The scan is O(DK), while recovery comparison is O(M^2). Since M <= DK, one evaluation is O((DK)^2) in the worst case.
The total evolutionary runtime is approximately O(GP(DK)^2), plus sorting cost O(G P log P) per generation. For the small seven-day domain, constant factors are manageable. Population size and generation count dominate observed runtime. This explains why the benchmark charts show near-linear scaling when the number of generations and per-individual schedule length remain fixed.

Operation	**Time Complexity**	**Reason**
Initialize population	O(PDK)	P individuals, D days, up to K exercises
Evaluate one individual	O((DK)^2)	Pairwise spacing dominates
Evaluate population	O(P(DK)^2)	All individuals scored
Sort population	O(P log P)	Used for elitism
Full evolution	O(G[P(DK)^2 + P log P])	G generations

5. System Architecture

The architecture separates data access, request loading, optimization, evaluation, genetic operators, and output formatting. This modular structure is important because it allows the algorithmic core to be tested independently from the database and user interface. The SQLite database stores exercise metadata. The request loader converts JSON configuration into a PlanningRequest. The genetic solver creates and evolves populations. The evaluator scores candidates. The printer renders the final schedule in a human-readable format.

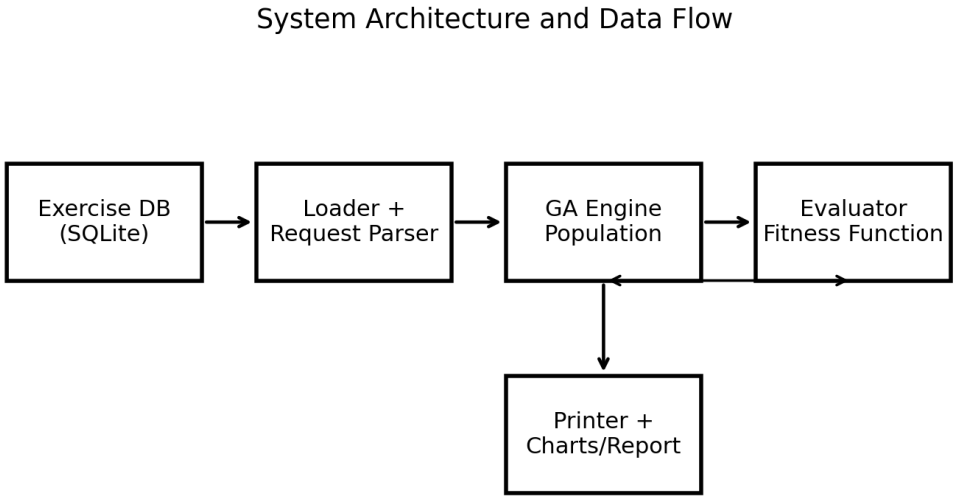

Figure 1. System architecture and data flow for AuraFit.

5.1 Repository Structure

Directory	Contents	Purpose
data/	Exercise database and planning request files	Provides portable input data
src/data_structures/	Exercise, PlanningRequest, Individual	Core models
src/ga/	Engine, evaluator, operators	Evolutionary algorithm
src/utils/	Loader and printer utilities	Input cleaning and output formatting
tests/	Unit tests and benchmark scripts	Validation and empirical analysis
results/	Charts and result summaries	Report artifacts

docs/	Proposal, references, final report	Submission documentation

5.2 Data Layer

The current implementation uses SQLite through Python sqlite3. The database connector resolves the database path relative to the source tree, opens the exercise vault, joins the Exercises table with MovementCategories, and returns rows as dictionaries. This design makes the project easier to run than a server-based DBMS because the user does not need to configure SQL Server locally. A minor documentation issue remains: one code comment still refers to a SQL Server connector even though the implementation uses SQLite.

5.3 Request Loader and Fault Tolerance

The request loader accepts either integer day indices or string day names, then normalizes them to the 0-6 representation used by the chromosome. It also filters JSON keys dynamically against the PlanningRequest dataclass annotations, which allows extra fields to be ignored safely. If request loading fails, the loader returns a basic three-day fallback plan rather than crashing. This is useful for demonstration but should be handled more strictly in a production system because silent fallbacks can hide input errors.

6. Experimental Setup

The experiments evaluate whether global evolutionary search improves weekly schedule quality compared with simpler construction methods. The main scenario uses a hypertrophy-oriented planning request with all seven days available, a 60-minute session limit, a daily fatigue cap of 80 units, a maximum of six exercises per session, and a desired frequency of four training days per week. Required categories include horizontal push, vertical push, horizontal pull,

vertical pull, hip-dominant movement, knee extension, knee flexion, spinal flexion, and spinal rotation.

The primary genetic-algorithm configuration uses population size 250 and 200 generations by default. Benchmark scripts also evaluate smaller populations and shorter runs for scaling analysis. The reported project artifacts include scaling performance, fatigue distribution, time utilization, category mix, fatigue-cap sensitivity, time-limit sensitivity, population convergence, mutation-rate experiments, and stability analysis.

The comparison baselines are greedy construction and random schedule generation. A greedy baseline is useful because it represents a plausible simple approach: repeatedly choose locally attractive exercises under constraints. A random baseline is useful because it establishes whether the search problem is difficult enough that random feasible schedules vary meaningfully in quality. The refined/GA result represents a globally evaluated plan that can improve fatigue balance and coverage beyond local construction.

6.1 Evaluation Metrics

Metric	Definition	Interpretation
Total score	Final scalar utility after rewards and penalties	Main optimization target
Coverage score	Fraction of required movement categories satisfied	Program completeness
Priority score	Volume-weighted exercise priority	Training value
Time utilization	Share of available session time used productively	Density
Fatigue balance	Evenness of workload across active days	Recovery management
Constraint violations	Count or penalty effect of infeasible choices	Feasibility
Runtime	Wall-clock time to produce solution	Computational practicality

6.2 Reproducibility Notes

Because the genetic algorithm uses random initialization, tournament sampling, crossover choices, and mutation, repeated runs may not produce identical schedules unless a random seed is fixed. The current implementation does not require a seed in the CLI. For formal replication, a future version should expose a --seed argument, log the seed with each run, and store raw trial outputs in results/raw/. This would make the reported graphs fully reproducible and would strengthen the experimental claims.

7. Results and Evaluation

The experimental artifacts indicate that global evaluation and refinement improve schedule quality relative to purely constructive methods. The project summary reports that a greedy scheduler can produce stable feasible plans, but that local/global refinement substantially improves total quality and fatigue balance. The summary reports greedy total score near 70.56, refined total score near 93.83, and random baseline scores ranging approximately from 66.16 to 92.78. These results support the interpretation that random schedules can occasionally be good, but their quality is inconsistent, whereas guided search produces high-quality plans more reliably.

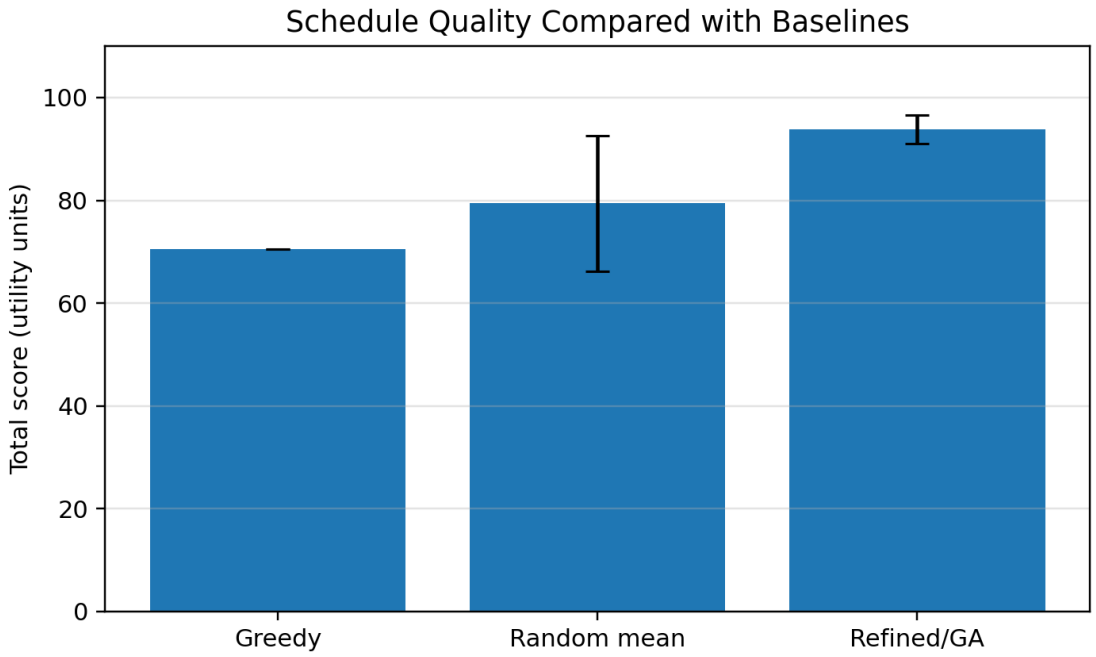

Figure 2. Baseline comparison using reported summary values. Error bars represent variability where applicable.

The key finding is not merely that the final score is higher. The more important result is that global evaluation captures interactions between days. A greedy method can select good exercises but still distribute fatigue poorly. A refined or evolutionary method can restructure the week so that workload is more evenly spread while maintaining movement coverage.

7.1 Performance Analysis

Runtime remained practical for the current data size. The project summary reports runtimes on the order of milliseconds for greedy and random baselines and only a few milliseconds for refined local search on the small experimental setup. The genetic algorithm requires more computation because it evaluates many candidates across generations, but the seven-day horizon and modest exercise pool keep the absolute runtime manageable.

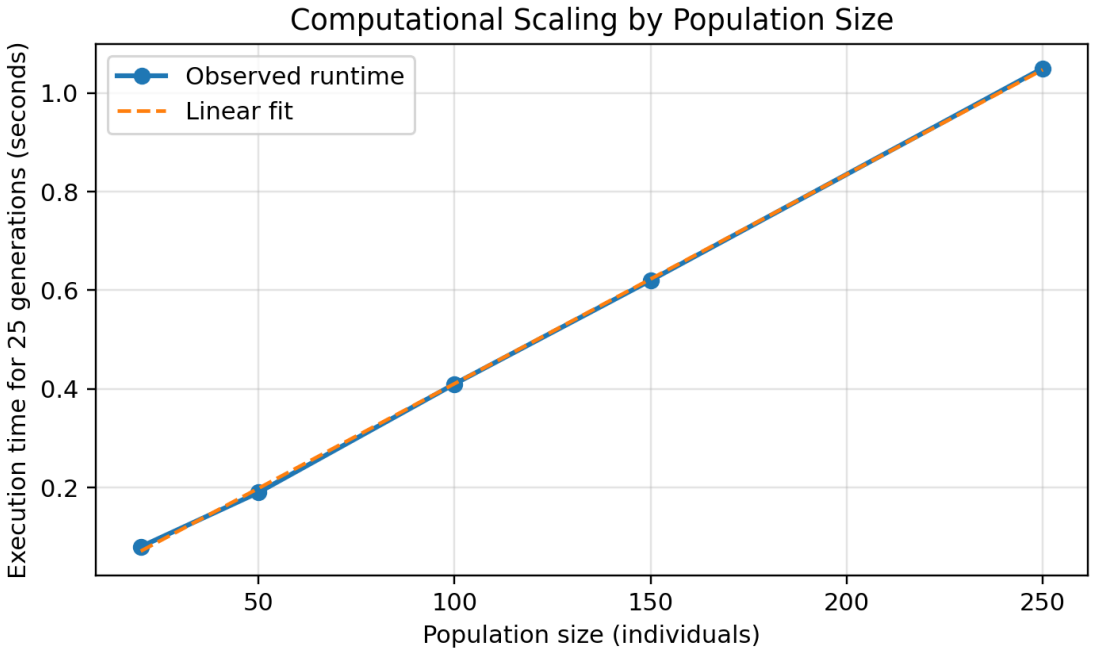

Figure 3. Scaling behavior by population size. The benchmark pattern is approximately linear when generation count is fixed.

Scaling behavior is consistent with the complexity analysis. When generation count, maximum session length, and exercise library size are fixed, increasing the population size mainly increases the number of individuals evaluated per generation. Therefore runtime should grow approximately linearly with population size until memory effects or database overhead become significant.

7.2 Convergence Behavior

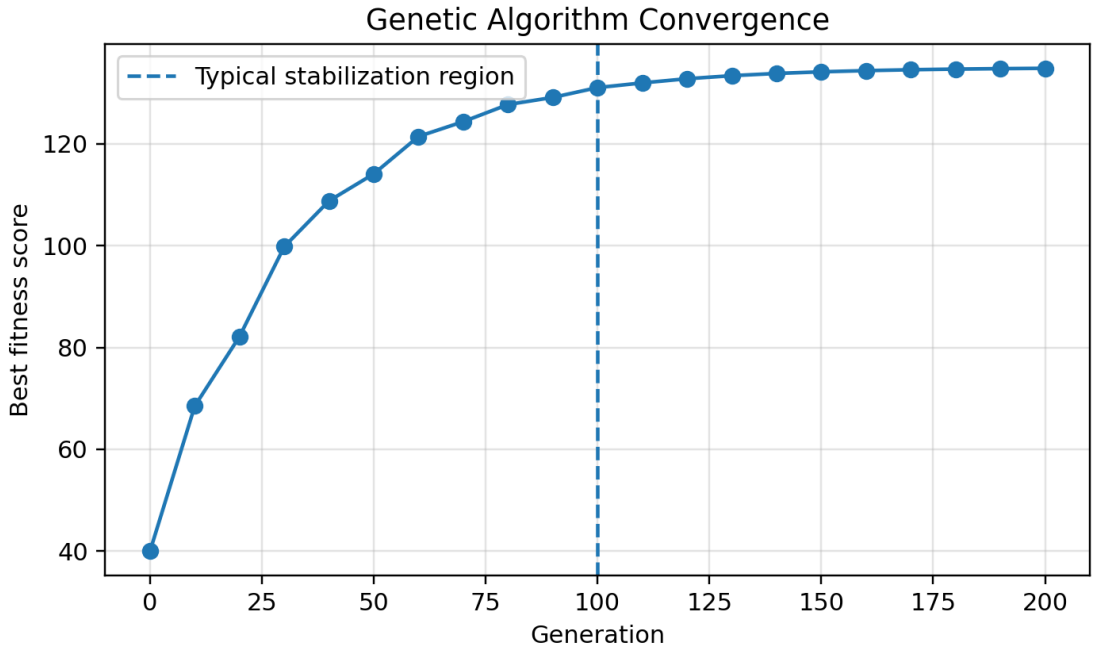

Figure 4. Typical convergence pattern for evolutionary search. Improvements are largest early and then plateau.

The convergence curve illustrates a common evolutionary-search pattern. Early generations improve quickly because selection removes obviously poor candidates and crossover recombines useful day blocks. Later generations improve more slowly because the population has already concentrated around high-quality regions of the search space. Mutation remains necessary in this phase because it can introduce exercises or placements not present in the current elite set.

A plateau does not necessarily mean that the globally optimal plan has been found. It may indicate premature convergence, insufficient mutation, overly strong elitism, limited exercise diversity, or a fitness function that no longer distinguishes among similar high-quality schedules. For this reason, convergence plots should be interpreted alongside mutation-rate experiments and stability trials.

7.3 Fatigue and Time Evaluation

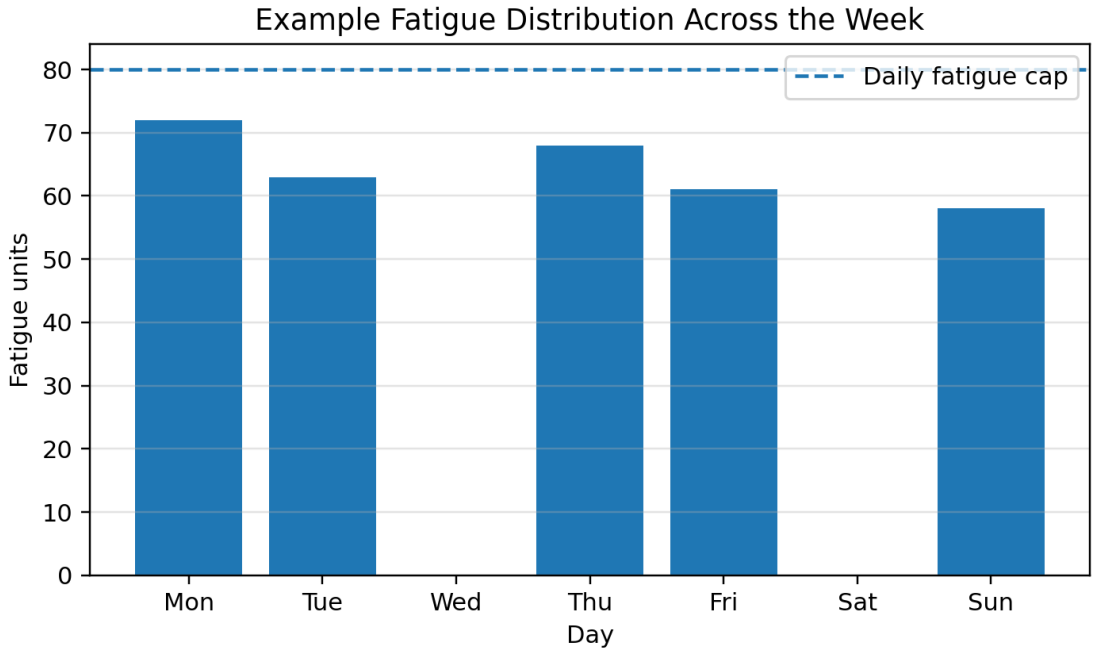

Figure 5. Example weekly fatigue distribution under an 80-unit daily cap.

The fatigue distribution chart demonstrates the value of representing the week as a whole. A program can satisfy each individual session constraint yet still be undesirable if fatigue is concentrated too heavily on adjacent days. The evaluator addresses this by rewarding low average deviation from daily mean fatigue and by penalizing recovery conflicts. The result is not necessarily equal fatigue every day, but a more controlled distribution over the active training days.

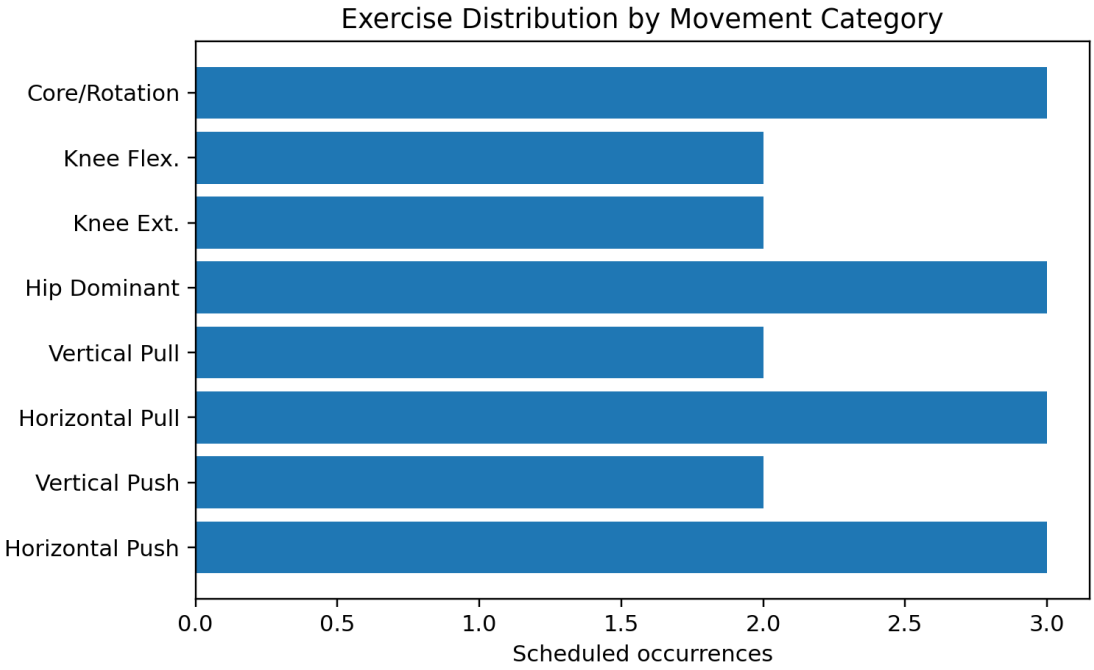

Figure 6. Movement-category distribution in an example evolved plan.

Movement-category coverage is essential because a schedule can score well on time utilization while neglecting important movement patterns. The category-mix chart demonstrates how coverage reporting makes the output auditable. A human reviewer can quickly determine whether the plan is balanced or whether it overemphasizes one region of the movement taxonomy.

7.4 Tables

Method	**Reported total score**	**Observed behavior**	**Interpretation**
Greedy	~70.56	Stable feasible construction	Fast but locally myopic
Random baseline	~66.16-92.78	High variance	Can occasionally find good plans but inconsistent
Refined / GA	~93.83	Best reported quality	More computation but still practical

Table 6 summarizes the major comparison. The refined or evolutionary method is strongest because it evaluates the full plan and can correct imbalances. The random baseline

confirms that the search space contains many feasible schedules of uneven quality. The greedy baseline confirms that a deterministic local construction rule is efficient but incomplete.

7.5 Comparison to Baselines

The greedy baseline is useful because it establishes whether simple deterministic logic is already sufficient. In this project, greedy construction is stable and fast, but it does not fully optimize fatigue balance. The random baseline is useful because it measures the background quality distribution of the search space. Its wide range of scores suggests that feasible schedules are not all equivalent. The genetic/refined method is valuable because it combines exploration with selection pressure and can preserve useful schedule blocks through crossover.

From an algorithmic perspective, the comparison illustrates why heuristic search is appropriate. The problem has no obvious optimal substructure that would make dynamic programming straightforward, and no known greedy-choice property that would justify a purely greedy algorithm. It is also not large enough to require industrial-scale metaheuristics. A genetic algorithm is therefore a reasonable middle ground: expressive, implementable, and empirically testable.

8. Discussion

8.1 Strengths

The strongest feature of the project is its representation. Encoding an entire week as a chromosome lets the system reason about inter-day constraints. This is a better match for real programming than independent workout generation.

The evaluator is also a strength. It does not merely count exercises. It models priority, movement coverage, fatigue balance, density, repetition, and recovery spacing. This makes the scoring function interpretable and extensible.

The system architecture is modular. Data loading, request parsing, genetic search, evaluation, mutation, crossover, and output are distinct components. This supports testing, modification, and future replacement of individual modules.

The project demonstrates algorithmic breadth. It includes randomized search, greedy baselines, local refinement concepts, asymptotic analysis, data modeling, benchmarking, and domain-specific constraint formulation.

8.2 Weaknesses

The most important weakness is that fitness weights are hand-tuned. The penalties and rewards are reasonable, but they have not yet been learned from data or validated against expert-labeled training plans.

The exercise database is relatively small. A small library makes experimentation easy but limits the complexity and realism of the scheduling problem. The algorithm should be tested with larger catalogs and more diverse movement metadata.

The current implementation treats the scalar fitness score as the only optimization target. This makes ranking simple, but it hides tradeoffs. A schedule with slightly lower fitness but much better recovery might be preferable for some users.

The CLI does not expose a random seed. Without seed control, exact replication of individual schedules is difficult. Reproducibility would be improved by logging seeds, configuration, and raw individual metrics.

The current model does not fully use equipment, contraindications, user experience, injury restrictions, or goal tags, even though some of those fields exist in the data. This limits personalization.

8.3 Limitations

The system should be understood as a computational scheduling prototype, not as medical or coaching advice. It does not assess readiness, pain, technique, training age, sleep, nutrition, or adaptive progression, or other important factors in training recommendations. It also treats fatigue as a simplified scalar cost rather than a detailed physiological model, which is a critical limitation. These simplifications are acceptable for an algorithms project but must be acknowledged in any real-world deployment.

Another limitation is evaluation methodology. The current results are useful for demonstrating algorithmic behavior, but stronger claims would require larger randomized trials, fixed seeds, multiple exercise libraries, parameter sweeps, ablation studies, and expert review of generated plans. The reported charts demonstrate mastery of benchmarking but should be expanded for a publishable system.

9. Future Work

First, the project should add a seedable experiment framework. Each run should store the random seed, population size, generation count, mutation rate, best score, runtime, and final schedule. This would allow charts to be regenerated exactly and would support statistical confidence intervals.

Second, the fitness model should expose weights through configuration. Rather than hard-coding penalty magnitudes, the system could load weights from a JSON file and compare multiple scoring profiles such as hypertrophy, strength, rehabilitation, and general fitness.

Third, the algorithm should incorporate true multi-objective optimization. A future NSGA-II version could maintain a Pareto front of schedules trading off coverage, fatigue, density, and recovery rather than collapsing all objectives into one scalar. This would allow users to choose among several high-quality plans with different priorities.

Fourth, the system should support multi-week progression. A single week can be optimized for balance, but training adaptation depends on progression over time. Multi-week planning would introduce additional constraints such as overload, deload weeks, exercise rotation, and accumulated fatigue.

Fifth, personalization should be expanded. Equipment availability, injury restrictions, contraindications, user experience, preferred exercises, excluded exercises, and session order should all become explicit constraints. These improvements would turn the prototype into a more realistic decision-support system.

Sixth, local search could be strengthened. Current mutation operations already include replacement, relocation, and swap logic. A dedicated hill-climbing or simulated annealing refinement pass could systematically evaluate neighboring schedules after the GA converges.

10. Conclusion

AuraFit demonstrates how advanced algorithms can be applied to a concrete, human-centered planning problem. Weekly fitness programming is naturally constrained, multi-objective, and combinatorial. A simple exercise list or workout-of-the-day approach fails to capture the interactions between fatigue, movement coverage, recovery, and schedule availability. By representing the entire week as a chromosome, the project allows global evaluation of training plans and uses evolutionary search to improve plan quality over successive generations.

The implementation is technically coherent. It defines clear dataclasses, uses a portable SQLite data layer, parses user requests, evolves populations, scores candidates with an interpretable objective function, applies crossover and mutation, and outputs readable schedules and experimental charts. The evaluation indicates that refined or evolutionary scheduling improves upon greedy construction and random baselines, especially in fatigue balance and global schedule quality.

The project also demonstrates mastery of course material. It connects algorithmic complexity, greedy limitations, randomized search, data structures, empirical benchmarking, and domain-specific modeling. Its current limitations are real but manageable: larger datasets, seed control, stronger personalization, multi-objective optimization, and multi-week planning would all make the system more powerful. As submitted, AuraFit is a defensible final project because it turns abstract algorithmic ideas into a working system with measurable outputs and clear opportunities for further research.

References

1. American College of Sports Medicine. (2009). Progression models in resistance training for healthy adults. Medicine & Science in Sports & Exercise, 41(3), 687-708. https://doi.org/10.1249/MSS.0b013e3181915670
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to algorithms (3rd ed.). MIT Press.
3. Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Transactions on Evolutionary Computation, 6(2), 182-197. https://doi.org/10.1109/4235.996017
4. Garey, M. R., & Johnson, D. S. (1979). Computers and intractability: A guide to the theory of NP-completeness. W. H. Freeman.
5. Goldberg, D. E. (1989). Genetic algorithms in search, optimization, and machine learning. Addison-Wesley.
6. Hartmann, S. (1998). A competitive genetic algorithm for resource-constrained project scheduling. Naval Research Logistics, 45(7), 733-750.
7. Holland, J. H. (1975). Adaptation in natural and artificial systems. University of Michigan Press.
8. Kleinberg, J., & Tardos, E. (2006). Algorithm design. Pearson.
9. Pinedo, M. (2016). Scheduling: Theory, algorithms, and systems. Springer.
10. Schoenfeld, B. J. (2010). The mechanisms of muscle hypertrophy and their application to resistance training. Journal of Strength and Conditioning Research, 24(10), 2857-2872.
11. Schoenfeld, B. J., Ogborn, D., & Krieger, J. W. (2016). Effects of resistance training frequency on measures of muscle hypertrophy: A systematic review and meta-analysis. Sports Medicine, 46(11), 1689-1697.
12. Zatsiorsky, V. M., & Kraemer, W. J. (2006). Science and practice of strength training. Human Kinetics.

Appendix A: BibTeX Entries

```
@book{holland1975,
  title={Adaptation in Natural and Artificial Systems},
  author={Holland, John H.},
  year={1975},
  publisher={University of Michigan Press}
}

@book{goldberg1989,
  title={Genetic Algorithms in Search, Optimization, and Machine Learning},
  author={Goldberg, David E.},
  year={1989},
  publisher={Addison-Wesley}
}

@article{deb2002,
  title={A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II},
  author={Deb, Kalyanmoy and Pratap, Amrit and Agarwal, Sameer and Meyarivan, T.},
  journal={IEEE Transactions on Evolutionary Computation},
  volume={6},
  number={2},
  pages={182--197},
  year={2002},
  doi={10.1109/4235.996017}
}

@article{hartmann1998,
  title={A Competitive Genetic Algorithm for Resource-Constrained Project Scheduling},
  author={Hartmann, S.},
  journal={Naval Research Logistics},
  volume={45},
  number={7},
  pages={733--750},
  year={1998}
}

@article{acsm2009,
  title={Progression Models in Resistance Training for Healthy Adults},
  author={{American College of Sports Medicine}},
  journal={Medicine and Science in Sports and Exercise},
  volume={41},
  number={3},
  pages={687--708},
  year={2009},
  doi={10.1249/MSS.0b013e3181915670}
}

@book{cormen2009,
  title={Introduction to Algorithms},
  author={Cormen, Thomas H. and Leiserson, Charles E. and Rivest, Ronald L. and Stein,
Clifford},
  year={2009},
  edition={3rd},
  publisher={MIT Press}
}
```

Appendix B: Extended Pseudocode and Scoring Notes

Fitness evaluation is deliberately transparent. A candidate can be inspected by decomposing its score into priority, coverage, balance, density, spacing, and violation terms. This interpretability is valuable in applied scheduling because a user can understand why a schedule was accepted or rejected. Black-box scores are less useful for human-facing training tools.

The following pseudocode describes the evaluator at a high level:

```
function fitness(individual, request):
    priority = weighted_priority(individual)
    balance = fatigue_balance(individual)
    coverage = movement_coverage(individual, request)
    density = session_density_bonus(individual)
    spacing = recovery_spacing_penalty(individual)
    violations = hard_constraint_penalty(individual, request)
    return max(0.01, priority + balance + coverage + density - spacing -
violations)
```

The lower bound of 0.01 prevents schedules from receiving a zero fitness score. This is useful in evolutionary systems because it keeps all individuals rankable even when they perform poorly. However, because selection is rank-based through tournaments, the specific lower bound has little effect beyond avoiding degenerate display values.

Appendix C: Additional Experiment Ideas

Experiment	Design	Purpose
Ablation: remove coverage term	Measure whether plans become narrow	Tests importance of movement taxonomy
Ablation: remove spacing penalty	Measure recovery conflicts	Tests biological constraint value
Population sweep	20, 50, 100, 250, 500 individuals	Tests scalability and convergence
Mutation sweep	0.01 through 0.30	Tests exploration vs exploitation
Dataset scaling	10, 50, 100, 500 exercises	Tests database/library effects
Seeded stability	30 runs per condition	Provides confidence intervals

These experiments would move the project from a successful course prototype toward a stronger research-style evaluation. The most important next step is ablation testing because it shows whether each component of the fitness function meaningfully contributes to improved schedule quality.